

第三章 RISC-V汇编及其指令系统

- RISC-V概述
- RISC-V汇编语言
- **RISC-V指令表示**
- 实例分析



第三章 RISC-V汇编及其指令系统

- **RISC-V指令表示**
 - RISC-V六种指令格式
 - RISC-V寻址模式
 - 立即数寻址
 - 寄存器寻址
 - 基址寻址（偏移寻址）
 - PC相对寻址



身份证号字段含义

340823199506177523

位数	内容	含义	解释
1-2	34	所在省份代码	表示安徽省
3-4	08	所在城市代码	表示安庆市
5-6	23	所在区县代码	表示枞阳县
7-14	19950617	出生年月日	表示1995年6月17日出生
15-16	75	出生顺序号	区域内75号
17	2	性别	奇数为男，偶数为女
18	3	身份证校验码	编制单位按公式计算，可为X

参考<https://zhidao.baidu.com/question/1613623520435376587/answer/3996411661.html?fr=zywd>

我校本科生学号字段含义

190110912

位数	内容	含义	解释
1-2	19	入学年份	表示2019年入学
3-4	01	学院代号	表示计算机科学与技术学院
5	1	专业代号	表示计算机专业
6-7	09	班级号	表示所在班级为9班
8-9	12	学号	表示学号为12号

指令格式

- 用二进制代码表示指令的结构形式
 - 要求计算机处理**什么数据**?
 - 要求计算机对数据做**什么处理**?
 - 计算机**怎样**才能**得到**要处理的**数据**?
- 通读黑书2.16-2.17, 了解不同ISA的指令格式差异

RISC-V指令表示

- 指令(字)分成多个“字段” (一部分位)
- 每个字段有特定的含义和作用
 - 理论上可以为每条指令定义不同的字段
- RISC-V定义了六种基本类型的指令格式:
 - R型指令 用于寄存器 - 寄存器操作
 - I型指令 用于短立即数和访问内存的 load 操作
 - S型指令 用于访问内存的 store 操作
 - B型指令 用于条件跳转/分支/转移操作
 - U型指令 用于长立即数操作
 - J型指令 用于无条件跳转操作

RISC-V指令类型

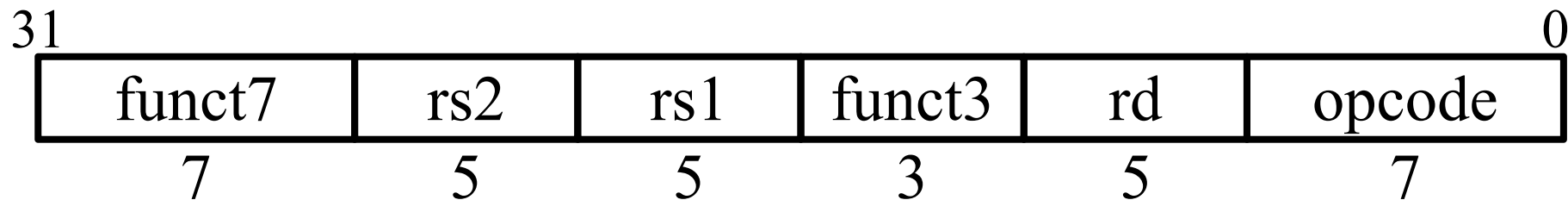
- **R-type**: **R**egister-**r**egister arithmetic/logical operations
- **I-type**: register-**I**mmEDIATE arithmetic/logical operations & **loads**
- **S-type**: **S**tore
- **B-type**: **B**ranches(**SB**型?)
- **U-type**: 20-bit **U**pper immediate instructions
- **J-type** : **J**umps (**UJ**型?)

RISC-V指令格式

- 会查表

	inst[31:25]	inst[24:20]	inst[19:15]	inst[14:12]	inst[11:7]	inst[6:0]
R型	func7	rs2(编号)	rs1(编号)	func3	rd(编号)	opcode
I型	imm[11:5]	imm[4:0]	rs1	func3	rd	opcode
S型	imm[11:5]	rs2	rs1	func3	imm[4:0]	opcode
B型	imm[12,10:5]	rs2	rs1	func3	imm[4:1,11]	opcode
J型	imm[20,10:5]	imm[4:1,11]	imm[19:12]		rd	opcode
U型	imm[31:12]				rd	opcode

R型指令

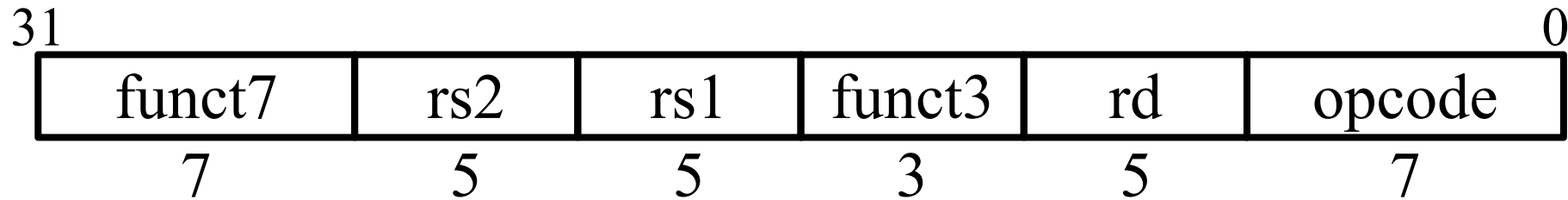


- 32位指令字，分为6个不同位数的字段
- opcode(操作码): 是哪类指令(inst[6:5] inst[4:2] inst[1:0])
- funct7+funct3: 这两个字段结合操作码确定要执行的操作

inst[4:2]	000	001	010	011	100	101	110	111
inst[6:5]								(> 32b)
00	LOAD	LOAD-FP	custom-0	MISC-MEM	OP-IMM	AUIPC	OP-IMM-32	48b
01	STORE	STORE-FP	custom-1	AMO	OP	LUI	OP-32	64b
10	MADD	MSUB	NMSUB	NMADD	OP-FP	reserved	custom-2/rv128	48b
11	BRANCH	JALR	reserved	JAL	SYSTEM	reserved	custom-3/rv128	≥ 80b

Table 24.1: RISC-V base opcode map, inst[1:0]=11

R型指令

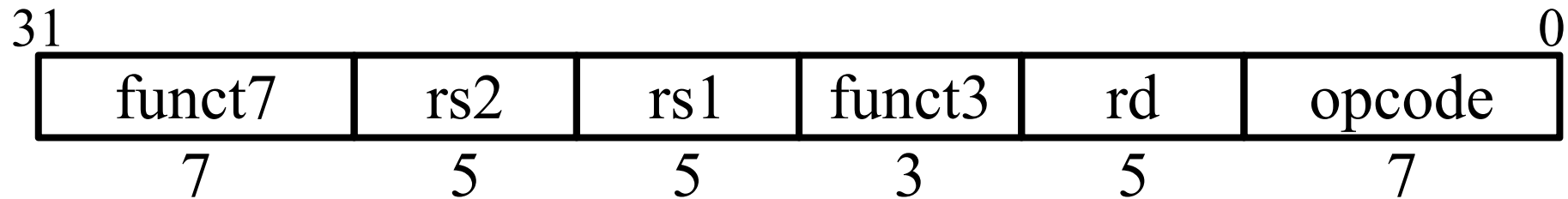


- rs1 (源寄存器1) : 指定第一个寄存器操作数
- rs2 (源寄存器1) : 指定第二个寄存器操作数
- rd (目的寄存器) : 指定接收计算结果的寄存器
- 每个寄存器字段是一个5位无符号整数 (对应寄存器编号)
- op rd, rs1, rs2

R型指令格式示例

请写出 `add x18, x19, x10` 对应的机器码

rd rs1 rs2



rs2=10 rs1=19

rd=18

查表确定 `add`指令对应的opcode、funct7、funct3

0000000	01010	10011	000	10010	0110011
---------	-------	-------	-----	-------	---------

add x8, sp, zero 对应的机器指令码为_____。

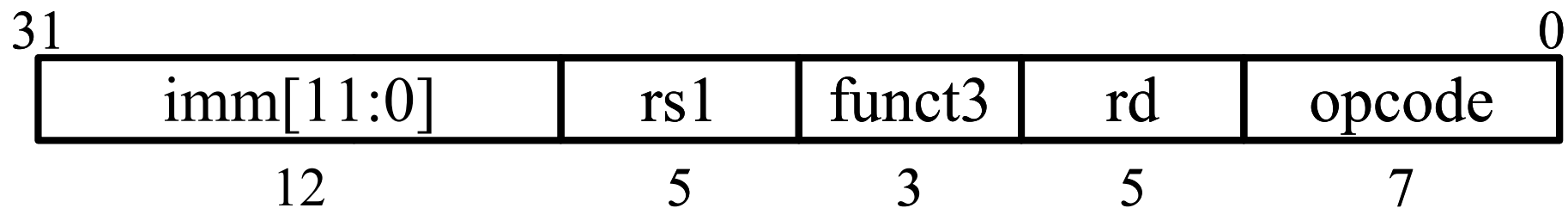
- ☐ A 0000 0000 1000 0001 0000 0000 0011 0011
- ☒ B 0000 0000 0000 0001 0000 0100 0011 0011
- ☐ C 0000 0000 1000 0001 0000 0000 0011 0011
- ☐ D 0000 0000 1000 0010 0000 0000 0011 0011

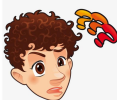
提交

R型指令

00000000	rs2	rs1	000	rd	0110011	add
0 1 000000	rs2	rs1	000	rd	0110011	sub
00000000	rs2	rs1	001	rd	0110011	sll
00000000	rs2	rs1	010	rd	0110011	slt
00000000	rs2	rs1	011	rd	0110011	sltu
00000000	rs2	rs1	100	rd	0110011	xor
00000000	rs2	rs1	101	rd	0110011	srl
0 1 000000	rs2	rs1	101	rd	0110011	sra
00000000	rs2	rs1	110	rd	0110011	or
00000000	rs2	rs1	111	rd	0110011	and

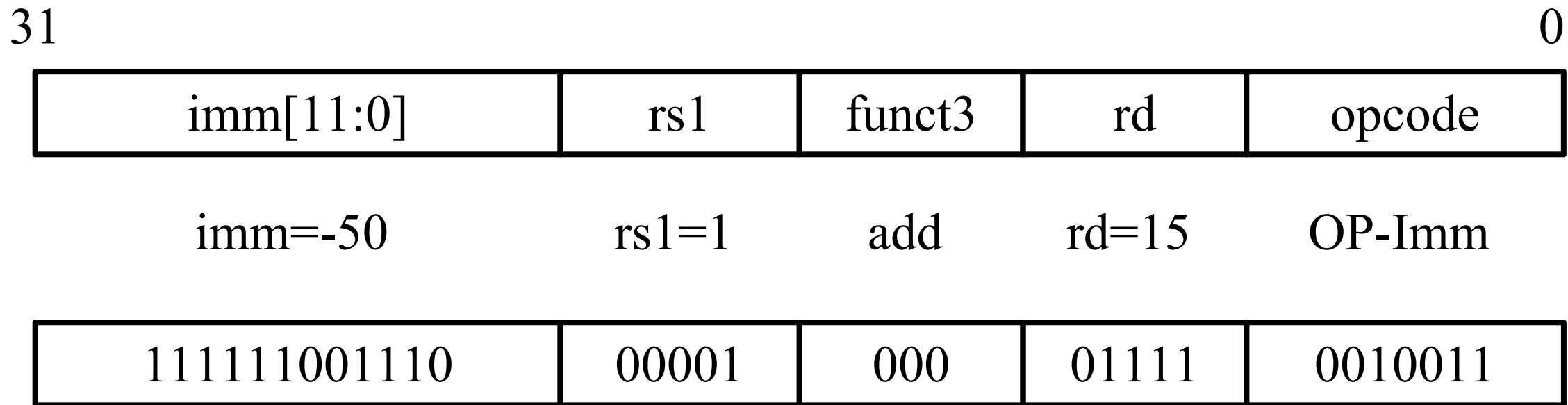
I型指令



- I型指令的rs1、funct3、rd、opcode字段与R型**相同**
- rs2和funct7被**12位有符号**立即数imm[11:0]替换
 - 立即数的表示范围? 
- 在算术运算前，立即数总是进行符号扩展

I型指令示例

addi x15, x1, -50



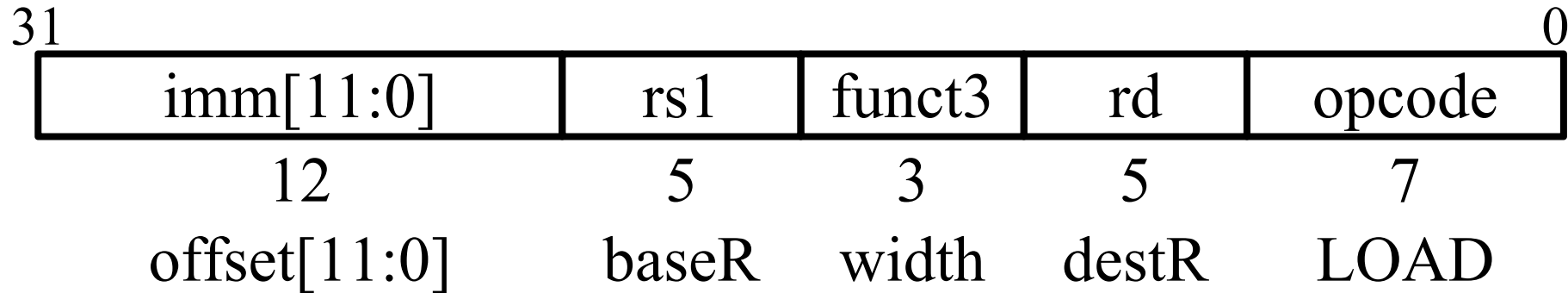
I型指令

imm[11:0]		rs1	000	rd	0010011	addi
imm[11:0]		rs1	010	rd	0010011	slti
imm[11:0]		rs1	011	rd	0010011	sltiu
imm[11:0]		rs1	100	rd	0010011	xori
imm[11:0]		rs1	110	rd	0010011	ori
imm[11:0]		rs1	111	rd	0010011	andi
000000	shamt	rs1	001	rd	0010011	slli
000000	shamt	rs1	101	rd	0010011	srli
010000	shamt	rs1	101	rd	0010011	srai

一个高立即数位用于区分逻辑右移(SRLI)和算术右移(SRAI)

RV64I中仅将立即数低6位shamt用作移位量(最多移位63次)

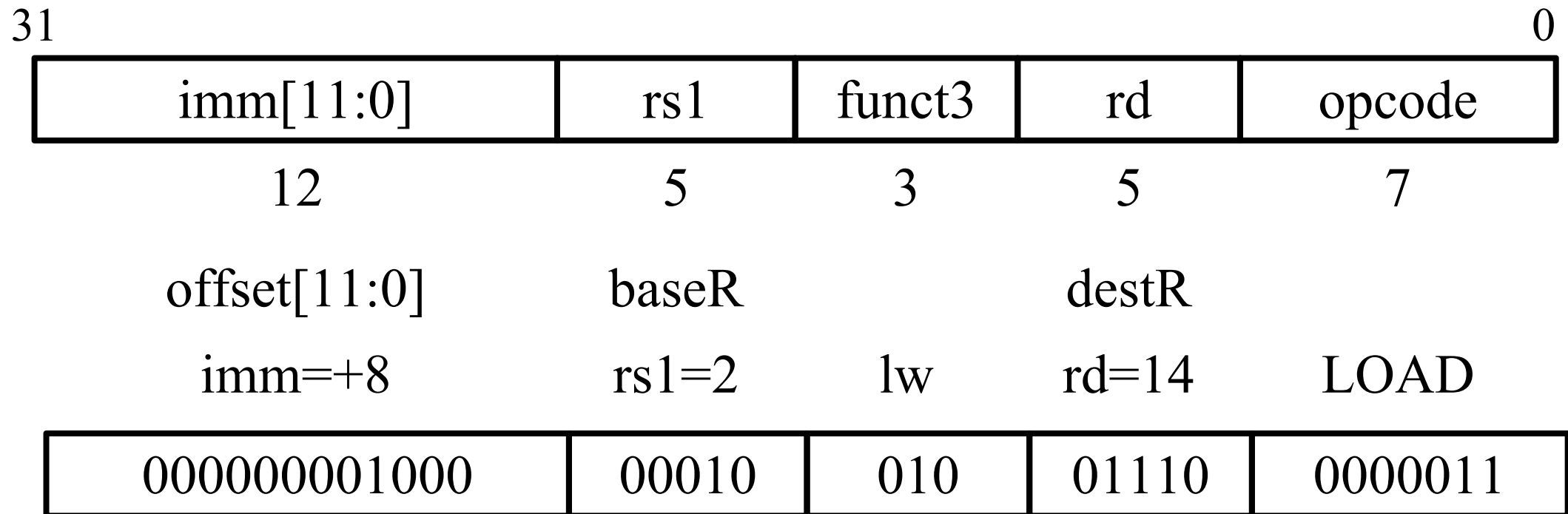
I型——Load类指令



- Load类指令也是I型指令
- 12位有符号立即数加寄存器rs1中的基址，形成内存地址
 - 与add immediate操作非常相似，但用于**创建地址**
- 从内存读取到的值存储在寄存器rd中
- width表示装载的数据位数宽度和是否考虑符号

Load类指令(I型)示例

lw x14, 8(x2)



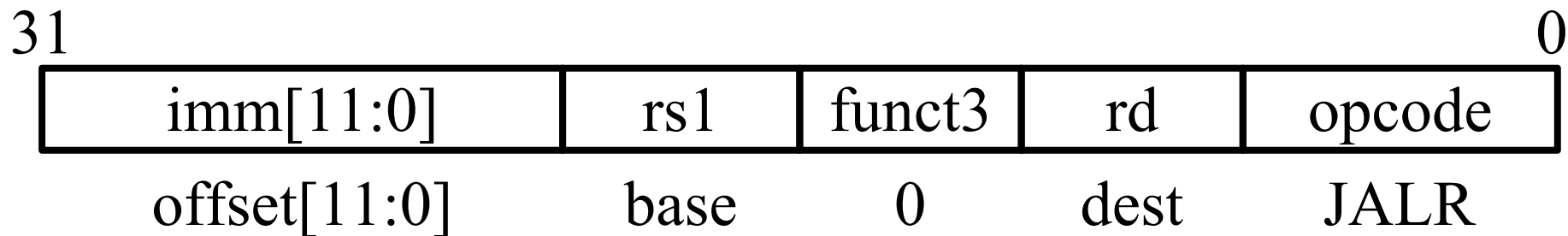
I型指令——load类指令列表

imm[11:0]	rs1	000	rd	0000011	lb
imm[11:0]	rs1	001	rd	0000011	lh
imm[11:0]	rs1	010	rd	0000011	lw
imm[11:0]	rs1	011	rd	0000011	ld
imm[11:0]	rs1	100	rd	0000011	lbu
imm[11:0]	rs1	101	rd	0000011	lhu
imm[11:0]	rs1	110	rd	0000011	lwu

funct3字段对读取**数据位宽**、**是否有符号** 进行编码

- LBU是“读取无符号字节”
- LH是“读取半字”，**符号扩展**以填充32/64位寄存器
- LHU是“读取无符号半字”，**零扩展**以填充32/64位寄存器

I型指令——jalr指令



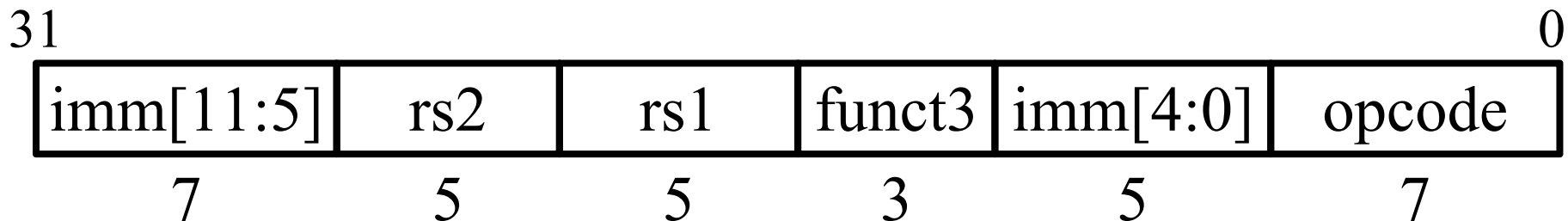
- jalr rd, offset(rs1) #(jump and link register)
- 将PC + 4保存在rd中
- 设置 PC = rs1 中的值 + offset
- 与load指令得到地址的方式类似

R型指令格式中，从右到左，依次为 [填空1]，[填空2]，[填空3]，[填空4]，[填空5]，[填空6]（填写rd,rs1,func3,opcode之类的字段名称）
 从右到左各个字段的长度分别为 [填空7]，[填空8]，[填空9]，[填空10]，[填空11]，[填空12]（填写位宽）

--	--	--	--	--	--

作答

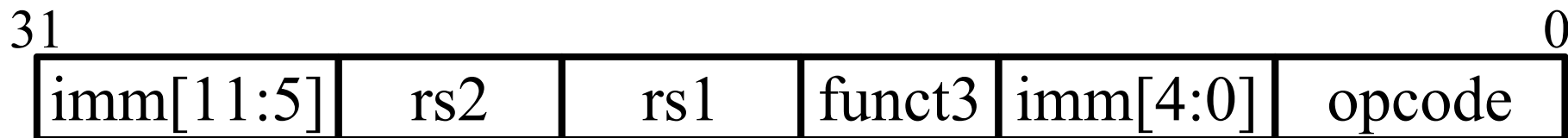
S型指令



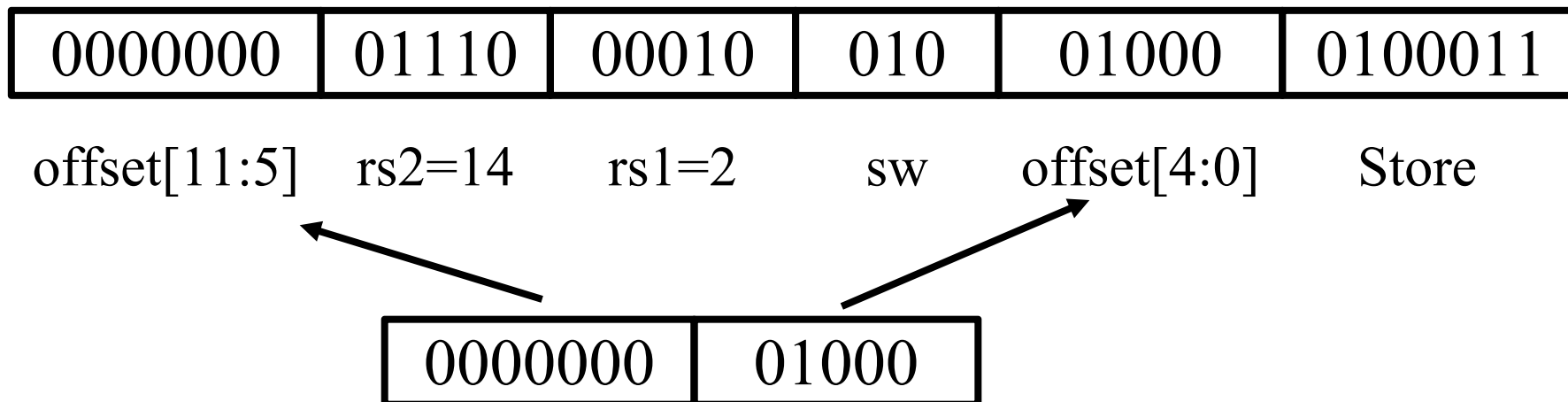
- **memop reg, offset(bAddrReg)**
- 存储指令需要读取两个寄存器，rs1中为内存地址，rs2中为要写入的数据；offset为立即数形式的地址偏移量
- S型指令不会将值写入寄存器，所以没有rd
- RISC-V的设计决定是将低位的5位立即数移到其他指令中的rd字段所在的位置——**保持rs1/rs2字段在同一位置**

RISC-V S型指令的机器表示

sb, sh, sw, sd



sw x14, 8(x2)



RISC V的S型指令

imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	sb
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	sh
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	sw
imm[11:5]	rs2	rs1	011	imm[4:0]	0100011	sd

- 黑书图2-18中sd的func3不对。封底中sd的类型不对，不是I型而是S型。 (RV spec 2019 P131)

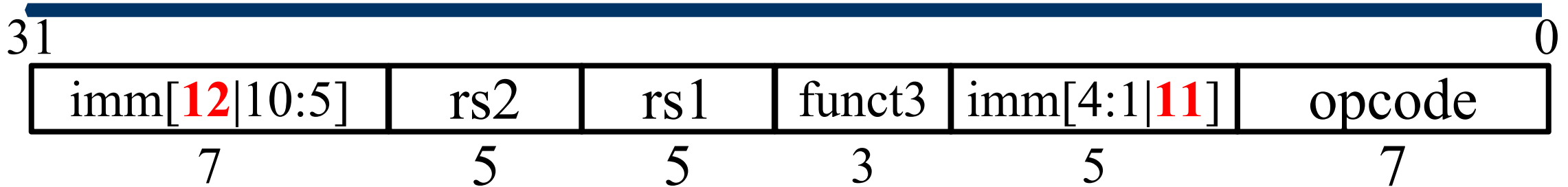
RV64I Base Instruction Set (in addition to RV32I)

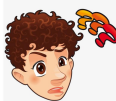
imm[11:0]		rs1	110	rd	0000011	LWU
imm[11:0]		rs1	011	rd	0000011	LD
imm[11:5]	rs2	rs1	011	imm[4:0]	0100011	SD

B型指令

- 主要用于条件分支或循环等
 - beq, bne, blt, bge, bltu, bgeu
- B型指令读取两个寄存器，但不写回寄存器
 - 类似于S型指令
- 指令格式中如何对Label进行编码？（用哪些字段）
 - 条件或循环体通常很小（<50条指令）
 - SPEC基准测试中约有一半条件分支跳转距离小于16条指令
 - 最大分支转移距离与代码量有关
- 如何根据指令格式计算转移的目标位置？

RISC-V中B型指令格式



- B型指令格式与S型指令格式**基本**相同(**imm[12:1]**)
- 立即数字段(12位)作为相对于**PC**的**补码**偏移量(**单位?**) 
 - 若偏移量为以**字节**为单位, 范围**约**为 $\pm 2^{11}$ 字节
 - 若偏移量为以**字** (指令) 为单位, 范围**约**为 $\pm 2^{11} \times 4$ 字节
- 基于RISC-V的扩展指令集支持16位压缩编码指令...
- 折中考虑, B型指令的偏移量以**半字**为单位(**2字节为增量**)
 - 偏移量范围**约**为 $\pm 2^{11} \times 2$ **字节** (-4096——4094**字节**)

例： 写出beq指令的机器码表示

Loop: **beq x19, x10, End**

add x18, x18, x10

addi x19, x19, -1

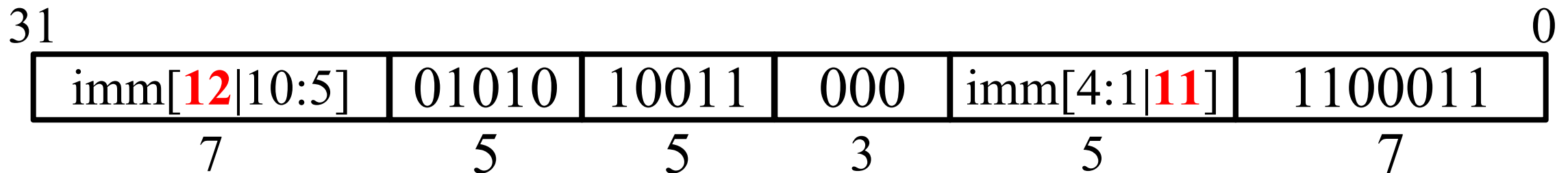
j Loop

End: # 目标指令

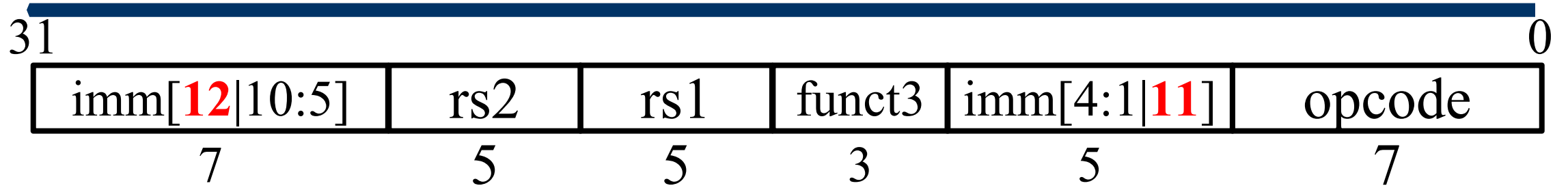


1 Count
2 instructions
3 from branch
4

offset = 4 words
= 16 bytes
= 8 × **2 bytes**

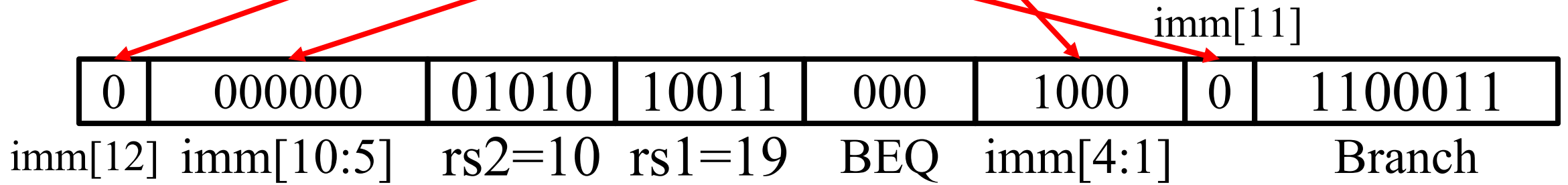
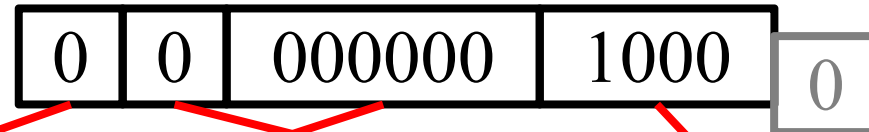


例中beq x19, x10, ...的立即数表示



beq x19, x10, offset = **8** (× 2 bytes)

imm[12][11][10 : 5] imm[4:1]



综合练习——黑书81页例题

- 对于以下汇编代码，假设循环的开始在内存80000处，那么这个循环的RISC-V机器代码是什么？

```
Loop: slli x10, x22, 3
      add x10, x10, x25
      ld x9, 0(x10)
      bne x9, x23, Exit
      addi x22, x22, 1
      beq x0, x0, Loop
```

Exit:

地址	指令
80000	00000000 00011 10110 001 01010 0010011
800	
800	
800	
800	
800	

B型指令列表

imm[12 10:5]	rs2	rs1	000	imm[4:1 11]	1100011	BEQ
imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011	BNE
imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011	BLT
imm[12 10:5]	rs2	rs1	101	imm[4:1 11]	1100011	BGE
imm[12 10:5]	rs2	rs1	110	imm[4:1 11]	1100011	BLTU
imm[12 10:5]	rs2	rs1	111	imm[4:1 11]	1100011	BGEU

RISC-V指令格式

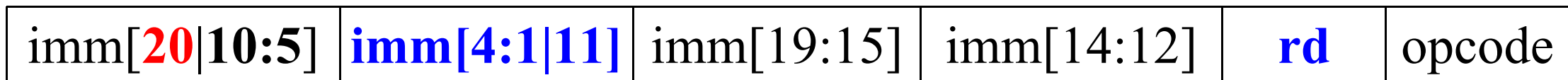
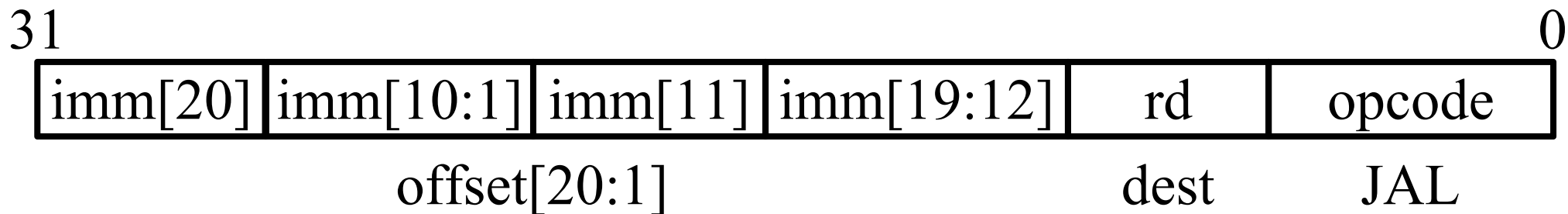
- 会查表

	inst[31:25]	inst[24:20]	inst[19:15]	inst[14:12]	inst[11:7]	inst[6:0]
R型	func7	rs2(编号)	rs1(编号)	func3	rd(编号)	opcode
I型	imm[11:5]	imm[4:0]	rs1	func3	rd	opcode
S型	imm[11:5]	rs2	rs1	func3	imm[4:0]	opcode
B型	imm[12,10:5]	rs2	rs1	func3	imm[4:1,11]	opcode
J型	imm[20,10:5]	imm[4:1,11]	imm[19:12]		rd	opcode
U型	imm[31:12]				rd	opcode

jal rd, offset # 将PC+4写入目的寄存器rd中

J型指令——jal

- jal rd, offset # 将PC+4写入目的寄存器rd中
 - j 是伪指令, j Label 相当于 jal x0, Label
- $PC = PC + offset$ (PC相对寻址)
- 访问相对PC,以**2字节**为单位的 $\pm 2^{19}$ 范围内的地址空间
 - $\pm 2^{18}$ **字**指令空间, 或 $\pm 2^{20}$ **字节 (1MiB)** 地址范围



U型指令

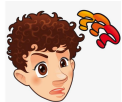


U-immediate[31:12]

dest

LUI

AUIPC

- ADDI等I型指令中的立即数范围是多少? 
- U型指令进行**长**立即数操作
 - 高20位为长度为20位的立即数字段
 - 5位目的地寄存器字段
- LUI(**L**oad **U**pper **I**mmediate)—将长立即数写入目的寄存器
- AUIPC(**A**dd **U**pper **I**mmediate to **PC**)——将PC与长立即数相加结果写入目的寄存器

U型指令——LUI(Load Upper Immediate)

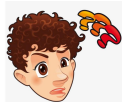
- LUI将**长**立即数写入目的寄存器的高**20**位，并**清除**低12位。
 - 立即数左移12位后写入目的寄存器
 - 对于RISC V64，**符号**扩展到64位。
 - LUI与ADDI一起可在寄存器中创建任何32位值
- LUI t0, 0x87654 #t0 = 0x87654**000**
- ADDI t0, t0, 0x321 # 本条语句执行后t0 = 0x87654321

Address	Code	Basic	Source
0x00400004	0x876542b7	LUI x5, 0x00087654	4: LUI t0, 0x87654 #t0 = 0x87654000
0x00400008	0x32128293	ADDI x5, x5, 0x00000321	5: ADDI t0, t0, 0x321 #t0 = 0x87654321

U型指令LUI的应用

- 如何创建 0xDEADBEEF?

LUI t2, 0xDEADB **B** # t2 = 0xDEADB **000**

#ADDI t2, t2, 0xEEF # **会出错**? too large to fit in ... 

ADDI t2, t2, 0xFFFFFEEF # t2 = 0xDEAD **A**EEF?

#ADDI 立即数总是进行**符号扩展**, 如果高位为1, 将从高位的20位中减去1

- 伪指令 li x10, 0xDEADB **B**EEF # 创建两条指令

LUI t1, 0xDEADC **C** # t1 = 0xDEADC000

ADDI t1, t1, 0xFFFFFEEF # t1 = 0xDEADBEEF

- **不用伪指令**如何创建64位立即数?

练习-U型指令

- 写出产生64位常量0x1122334455667B88的RISC-V汇编代码，并将该值存储到寄存器x10(RISC V 64)。

lui x10, 0x11223 #x10=0x0000 0000 1122 3000

addi x10, x10, 0x344 #x10=0x0000 0000 1122 3344

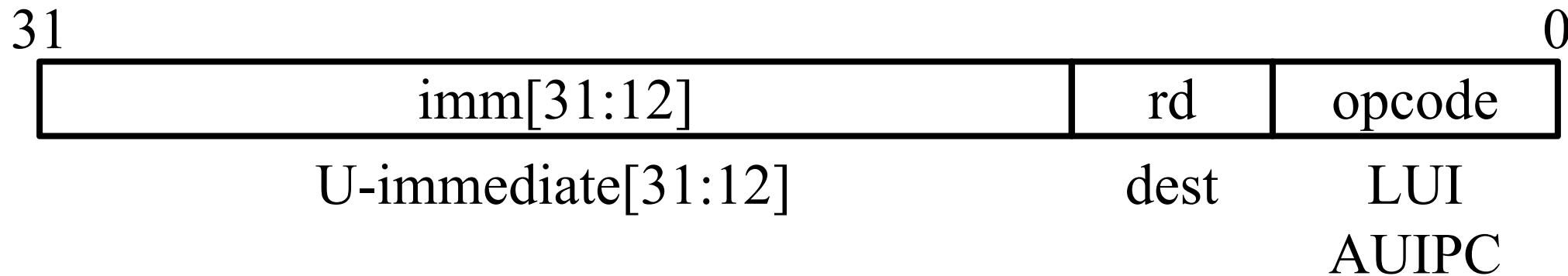
slli x10, x10, 32 #x10=0x1122 3344 0000 0000

lui x5, 0x5566**8** #x5 = 0x0000 0000 5566 **8**000

addi x5, x5, 0x**FFFFFF**B88 #x5 = 0x0000 0000 5566 **7B**88

add x10, x10, x5 #x10=0x1122 3344 5566 7B88

U型指令—AUIPC(Add Upper Immediate to PC)



- 将长立即数值加到PC并写入目的寄存器
- 用于PC相对寻址

Label: AUIPC rd, imm #将Label地址+立即数imm存rd

lui,auipc,jalr指令的应用

- ret 和 jr 伪指令

ret = jr ra = jalr x0, 0(ra)

- 对任意32位绝对地址处的函数进行调用

lui Reg1, <hi20bits> #<...>示意地址的高20位,%hi(label)

jalr rd, <lo12bits>(Reg1) #<lo12bits>地址的低12位

- 相对PC地址32位偏移量的相对寻址

auipc ra, <hi20bits>

jalr rd, <lo12bits>(ra)

RISC-V指令格式

- 立即数的**符号位**在最左边的高位
- 字段保持**规整性**

R型	func7	rs2	rs1	func3	rd	opcode
I型	imm[11 ,10:5]	imm[4:0]	rs1	func3	rd	opcode
S型	imm[11 ,10:5]	rs2	rs1	func3	imm[4:0]	opcode
B型	imm[12 ,10:5]	rs2	rs1	func3	imm[4:1, 11]	opcode
J型	imm[20 ,10:5]	imm[4:1, 11]	imm[19:15]	imm[14:12]	rd	opcode
U型	imm[31 :25]	imm[24:20]	imm[19:15]	imm[14:12]	rd	opcode

第三章 RISC-V汇编及其指令系统

- **RISC-V指令表示**

- RISC-V六种指令格式
- **RISC-V寻址模式** (确定操作数的方式)
 - 立即数寻址
 - 寄存器寻址
 - 基址寻址 (偏移寻址)
 - PC相对寻址



立即数寻址

- 操作数是指令本身包含的常量

addi x3, x4, **10**

andi x5, x6, **3**

immediate	rs1	funct3	rd	opcode
------------------	-----	--------	----	--------

寄存器寻址

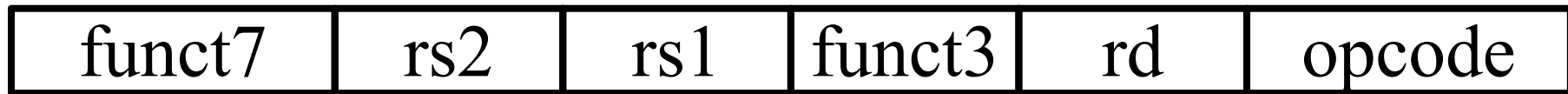
- 操作数在寄存器中

add x1, x2, x3

and x5, x6, x7

add*i* x3, x4, **10**

一条指令可以使用不止一种寻址方式



寄存器

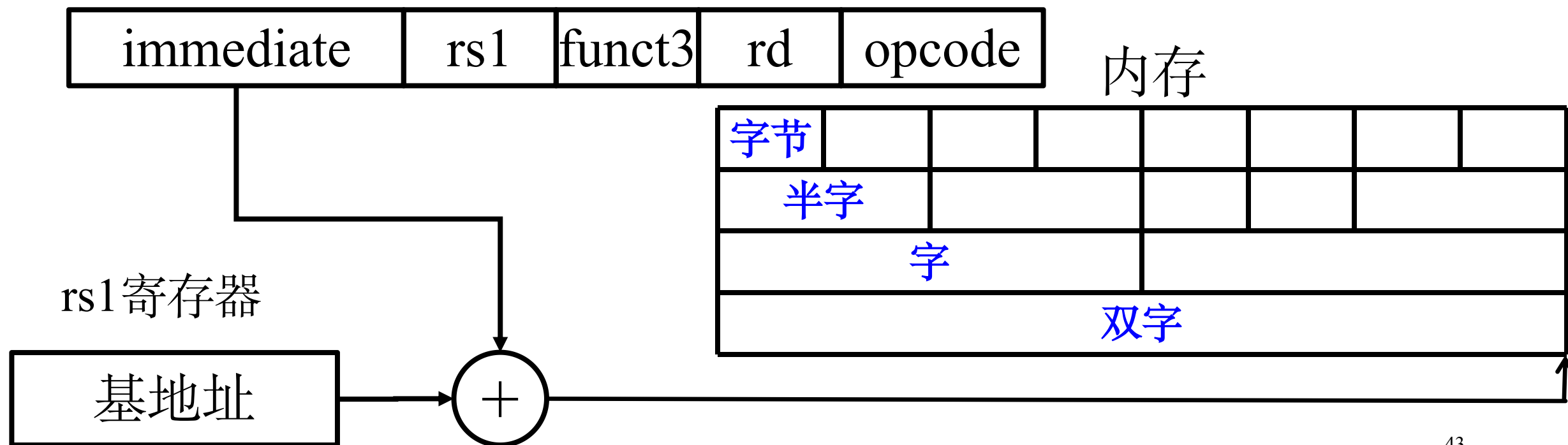
操作数

基址或偏移寻址

- 操作数在内存中，其地址是寄存器和指令中的常量之和

lw x10, 12(x15)

- 基址寄存器(x15)的数值 + 立即数偏移量(12)



练习——基址寻址

- 对于以下C语句，编写相应的RISC-V汇编代码。

`B [ 8 ] = A [ i - j ]; //long long int A[1024], B[1024];`

假设:为变量i和j分配寄存器x28和x29,A和B的基址分别在寄存器x10和x11中。

`sub x30, x28, x29` `#i-j`

`slli x30, x30, 3` `#8*(i-j)`

`add x30,x10,x30` `#&A[i-j] → x30`

`ld x30, 0(x30)` `#A[i-j] → x30` RARS中**直接执行会出错?**

`sd x30, 64(x11)` `#x30 → B[8]`

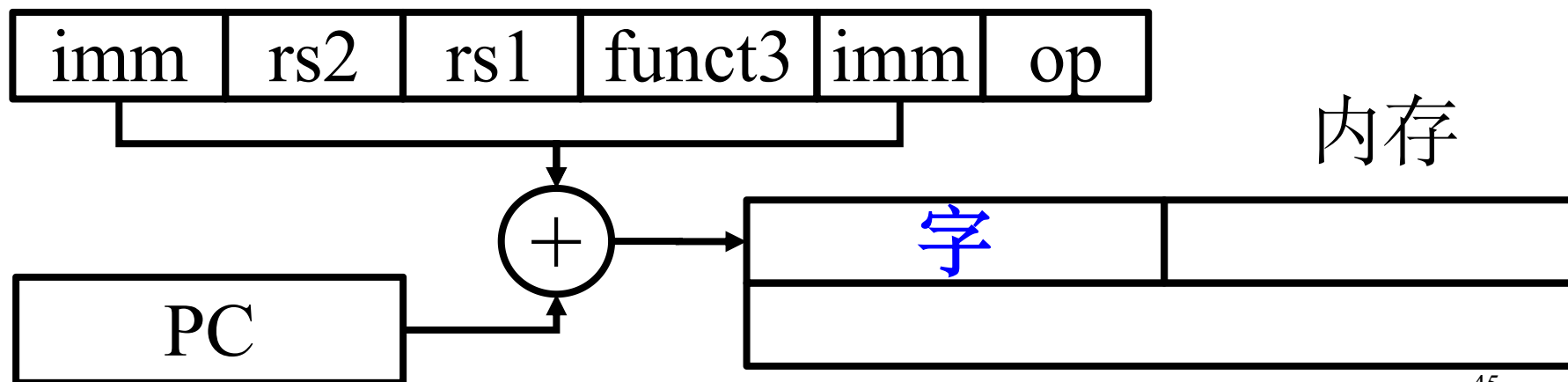
PC相对寻址

Loop: beq x19, x10, End
 add x18, x18, x10
 addi x19, x19, -1
 jal x0, Loop

1 Count
2 instructions
3 from branch
4

End:....

- 分支地址是PC与指令中立即数之和



第三章 RISC-V汇编及其指令系统

- 计算机中数的表示
- RISC-V概述
- RISC-V汇编语言
- RISC-V指令表示
- 实例分析



第三章 RISC-V汇编及其指令系统

- 实例分析
 - 数组与指针
 - RISC-V冒泡排序



两种数组清零方法的对比（黑书2.14）

用下标访问数组元素	用指针访问数组元素
<pre>clear1(long long int array[], int size){ int i; for(i=0; i<size; i+=1) array[i]=0; } //a0中存array首地址,a1中存size</pre>	<pre>clear2(long long int *array, int size){ long long int *p; for(p=&array[0]; p< &array[size];p=p+1) *p=0; } //a0中存array首地址,a1中存size</pre>
<pre>addi t0,zero,0 #i=0 loop1: slli t1,t0,3 #t1=i*8 add t2,a0,t1 #t2=array[i]地址 sd zero,0(t2) #array[i]=0 addi t0,t0,1 #i=i+1元素计数加1 blt t0,a1,loop1 #i<size, 循环</pre>	<pre>addi t0, a0, 0 #t0=array[0]地址 slli t1, a1,3 #t1=size*8 add t2,a0,t1 #t2=最后一个元素的地址 loop2:sd zero,0(t0) #memory[p]=0 addi t0,t0,8 #p=p+8 bltu t0,t2,loop2 #未到最后则循环</pre>

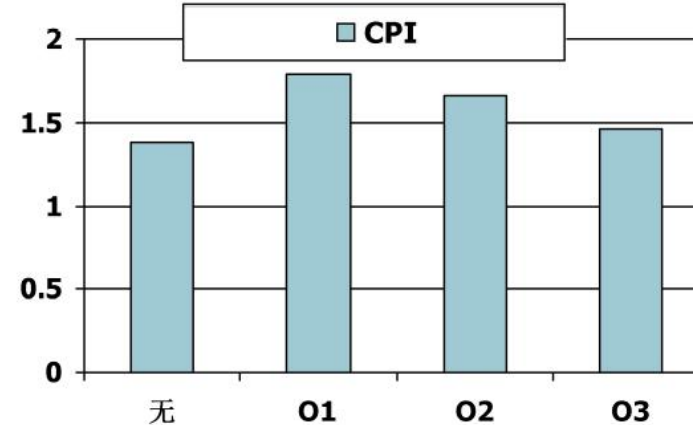
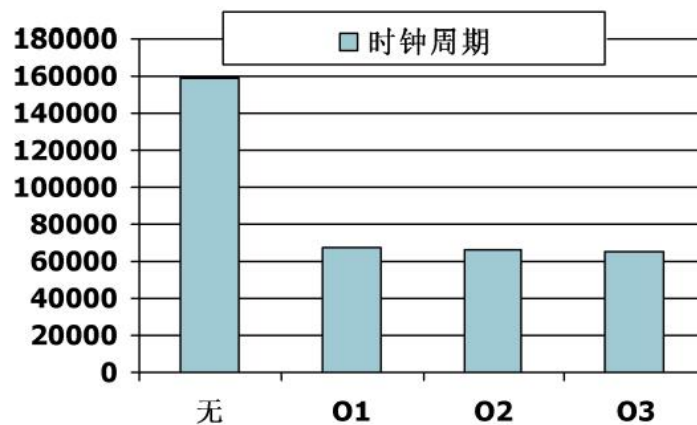
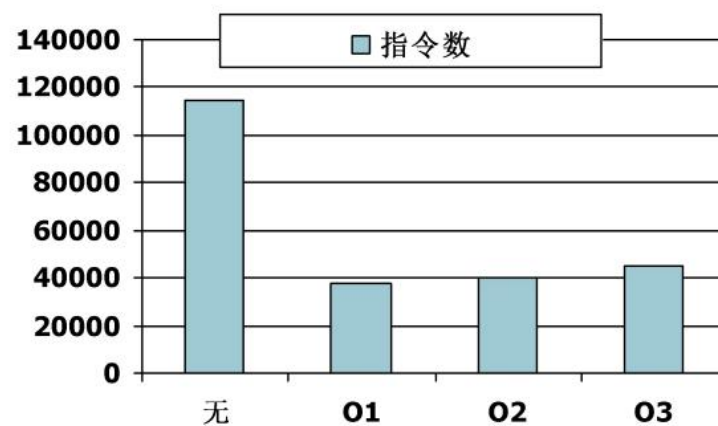
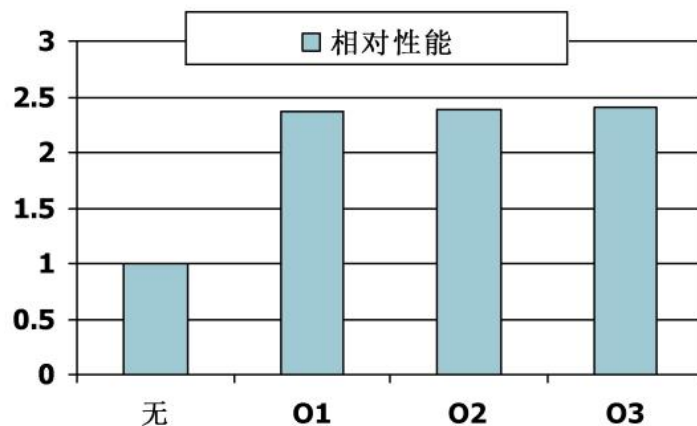
数组清零代码的分析

- 乘8——>左移3次 (编译器优化——强度削弱)
- 指针版本的循环体指令数少于数组版本
 - 数组版本i自增1后, 重新计算下一个数组元素的地址 (+8)
 - 指针版本的循环体内少了 $i*8$ 和求新元素地址
 - 编译器优化——循环变量消除
- 高级语言编程推荐使用下标方式访问数组元素
- 编译器优化生成的代码与手动使用指针一样有效

编译器优化的影响

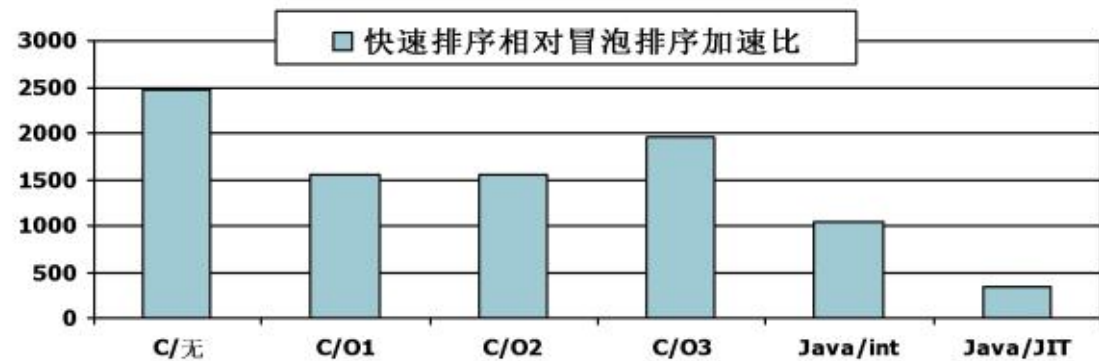
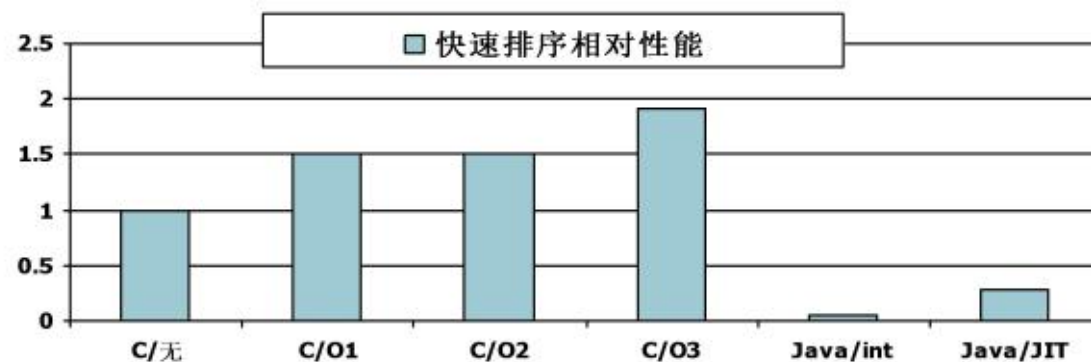
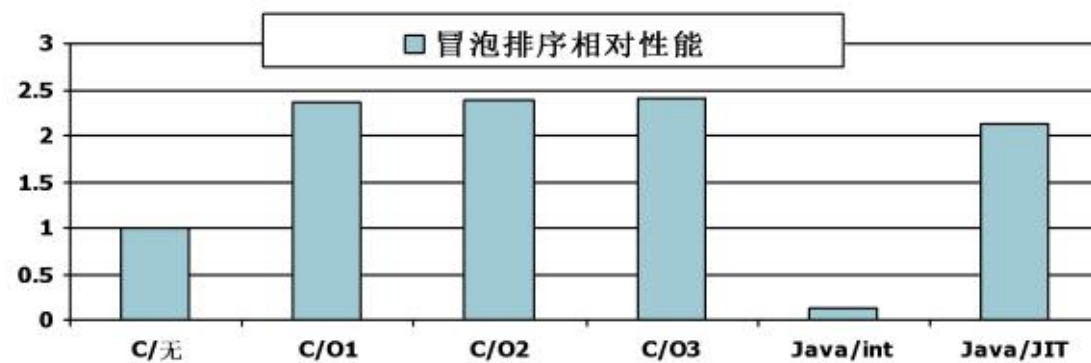
- 单看指令数或CPI都不是好的性能指标

用gcc for Pentium 4在Linux下编译



编程语言和算法的影响

- 编译器优化对算法敏感
 - 例如o1-o3对冒泡排序优化有限
- JIT代码明显快于JVM解释代码
 - 有时媲美优化过的C代码
- 没有什么可以弥补拙劣的算法!



第三章 RISC-V汇编及其指令系统

- RISC-V概述
- RISC-V汇编语言
- RISC-V指令表示
- 实例分析
 - 数组与指针
 - **RISC-V冒泡排序**



C语言排序函数的汇编实现（黑书2.13）

(C语言)swap过程（**叶过程**：不调用其他过程的过程）

```
void swap(long long int v[], long long int k) {  
    long long int temp;  
    temp = v[k];  
    v[k] = v[k+1];  
    v[k+1] = temp;  
}
```

RISC-V: 假设v保存在a0, k保存在a1, temp保存在t0

swap过程 (RISC-V64)

swap: #假设v保存在a0, k保存在a1, temp保存在t0

slli t1, a1, 3 # $t1 = k * 8$

add t1, a0, t1 # $t1 = v + (k * 8)$

ld t0, 0(t1) # (临时变量) $t0 = v[k]$

ld t2, 8(t1) # $t2 = v[k + 1]$

sd t2, 0(t1) # $v[k] = t2$

sd t0, 8(t1) # $v[k+1] = t0$ (临时变量)

jalr x0, 0(ra) # 返回调用函数

C版本的sort过程（非叶过程）

```
void sort (long long int v[], size_t n) {  
    size_t i, j;  
    for (i = 1; i < n; i += 1) {  
        for (j = i - 1; j >= 0 && v[j] > v[j + 1]; j -= 1) {  
            swap(v, j);  
        }  
    }  
}
```

- 假设v保存在x10, n保存在x11, i保存在x19, j保存在x20

外层循环的RV实现(for (i = 0; i <n; i += 1))

假设v保存在x10, n保存在x11, i保存在x19, j保存在x20

add x19,x0,x0 # i = 0

for1tst:

bge x19,x11,exit1 # 如果 $i \geq n$, 跳转到exit1

..... #外层循环的函数体, 包含内层循环

addi x19,x19,1 # i += 1

j for1tst # 跳转到外层循环的判断语句

exit1:

内层循环的RV实现

```
//for (j = i - 1; j >= 0 && v[j] > v[j + 1]; j - = 1) { ...}
    addi x20,x19,-1    # j = i - 1
for2tst:
    blt x20,x0,exit2    # 如果j < 0, 跳转到exit2
    slli x5,x20,3        # x5 = j * 8
    add x5,x10,x5        # x5 = v + (j * 8)
    ld x6,0(x5)          # x6 = v[j]
    ld x7,8(x5)          # x7 = v[j + 1]
    ble x6,x7,exit2      # 如果x6 ≤ x7, 转exit2
    mv x21, a0           # 将参数x10复制到x21
    mv x22, a1           # 将参数x11复制到x22
    mv a0, x21           # swap的第一个参数是v
    mv a1, x20           # swap的第二个参数是j
                                jal x1,swap        # 调用swap
                                addi x20,x20,-1    # j - = 1
                                j for2tst        #跳转到内层循环判断语句
exit2:...
```

假设v保存在x10, n保存在x11, i保存在x19, j保存在x20

RARS中可运行的汇编程序

C:\Codes\MyRV\strcpy-alias.asm - RARS 1.5

File Edit Run Settings Tools Help

Run speed at max (no interaction)

Edit Execute

strcpy-alias.asm

```
1  .data    0x10010000
2  x:
3      .byte  0
4      .space 13
5  y:
6      .byte  '0','1','2','3','4','5','6','F','a','d','f',0
7      .space 13
8
9  .text
10     lui   a0,0x10010    #目的字符串x首地址存于a0
11     #  lbu a0,0(s0)
12     addi  a1,a0,14      #源字符串y首地址存于a1
13     #  lbu a1,0(s1)
14     #la    x10,x
```

第三章 RISC-V汇编及其指令系统

- RISC-V汇编语言
 - RISC-V I 常用汇编指令要熟记
- RISC-V指令表示
 - 汇编指令和二进制机器指令互相转换（会查表）
 - 常用的六种指令格式
 - 四种常用的寻址方式

